

Implementation of each modules

(I didn't write about the modules that are already supplied)

Modules such as Adder, ALU_Control, ALU, Control, MUX32, Sign_Extend are still basically the same as the implementation in single cycle CPU. Only slight modifications were made to support more instructions such as sw, lw, and beq. To enable pipelining, I added pipeline register modules – IF_ID, ID_EX, EX_MEM, MEM_WB to store intermediate data between stages. I also added Branch_Unit, Forwarding_Unit, and Hazard_Detection to handle beq instructions, detect hazards, and reduce stall cycles.

- Adder.v

The Adder module performs a 32-bit addition. It takes two 32-bit inputs and outputs their sum. This module is used to compute PC + 4 for sequential instruction execution.

- ALU_Control.v

The ALU_Control module takes a 10-bit funct field (from funct7 and funct3) and a 2-bit ALUOp as input, then determines the ALUOp number first to see if it is an I-type, R-type, or branch instruction. Then, it determines funct to see which function it matches to, and passes the 3-bit result to the ALU. Then it outputs a 3-bit ALUCtrl_o to memorize which kind of ALU operation it is going to perform later in the ALU.

- ALU.v

The ALU module takes two 32-bit operands and a 3-bit control signal as input. The ALU performs the operation specified by the 3-bit control signal and outputs the result.

- Branch_Unit.v

The Branch Unit takes five inputs – RS1data_i, RS2data_i, Branch_i, ID_pc_i, imm32_i and two outputs – Flush_o, jump_addr_o.

It'd calculate and set jump_addr_o to ID_pc_i + (imm32_i << 1), then check if there is a branch instruction (which is beq in this case) and if RS1data_i == RS2data_i. If the condition holds, then the program has to jump and Flush_o would be set to 1.

- Control.v

The Control module takes opcode_i and noop_i as the inputs, then determines if it's an I-type, R-type, lw, sw, or beq instruction base on the opcode, and sets the seven outputs – ALUOp_o, ALUSrc_o, RegWrite_o, MemtoReg_o, MemRead_o, MemWrite_o, Branch_o accordingly.

- CPU.v

The CPU module is the top-level module that integrates all components. It defines the datapath and control logic needed to execute R-type, I-type, sw, lw, and beq instructions in pipeline cycles.

- EX_MEM.v

EX_MEM's inputs include: clk_i, RegWrite_i, MemtoReg_i, MemRead_i, MemWrite_i, RS2data_i, RDaddr_i, and ALUres_i. Its outputs include: RegWrite_o, MemtoReg_o, MemRead_o, MemWrite_o, RS2data_o, RDaddr_o, and ALUres_o. At every positive clock edge, the module updates its output registers to match the corresponding input values.

- Forwarding_Unit.v

Forwarding_Unit takes six inputs - EX_RS1addr_i, EX_RS2addr_i, MEM_RegWrite_i, MEM_RDaddr_i, WB_RegWrite_i, WB_RDaddr_i, and produces two outputs - ForwardA_o and ForwardB_o. Both outputs are initialized to 00. The module first checks for an EX hazard—if the condition is met, the corresponding output is set to 10. If not, it then checks for a MEM hazard; if that's true, the output is set to 01.

- Hazard_Detection.v

The Hazard_Detection module takes four inputs: RS1addr_i, RS2addr_i, EX_MemRead_i, and EX_RDaddr_i, and produces three outputs: NoOp_o, Stall_o, and PCWrite_o. By default, NoOp_o = 0, Stall_o = 0, and PCWrite_o = 1, meaning the pipeline is executing normally without any stalls. The module then checks for hazards caused by load instructions using the condition described in the slides. If a hazard is detected, the outputs are set to: NoOp_o = 1, Stall_o = 1, and PCWrite_o = 0.

- ID_EX.v

ID_EX has tons of inputs and outputs. Inputs include: clk_i, ALUOp_i, ALUSrc_i, RegWrite_i, MemtoReg_i, MemRead_i, MemWrite_i, RS1data_i, RS2data_i, RS1addr_i, RS2addr_i, funct_i, imm32_i, and RDaddr_i. Each input except clk_i has a corresponding output with the same name but ending in _o. At every positive clock edge, the module updates its output registers to match the corresponding input values.

- IF_ID.v

IF_ID takes five inputs – clk_i, Stall_i, Flush_i, instr_i, pc_i and produces two outputs – instr_o, pc_o. On every rising edge of clk_i, if Flush_i bit is 1, then both pc_o and instr_o are set to 0. Otherwise, if Stall_i is 0 (not gonna stall), pc_i is passed to pc_o and instr_i is passed to instr_o.

- **MEM_WB.v**
MEM_WB takes six inputs - clk_i, RegWrite_i, MemtoReg_i, ALUres_i, Memdata_i, RDAddr_i and produces five outputs - RegWrite_o, MemtoReg_o, ALUres_o, Memdata_o, RDAddr_o. The module update its output registers to match the corresponding input values at every postitive clock edge.
- **MUX32.v**
The MUX32 module is a 2-to-1 multiplexer. It takes two 32-bit inputs and a single-bit selector. If the selector is 1, the output is the second input; if 0, the output is the first input. This is used to choose between register data and immediate values for ALU input.
- **MUX32FU.v**
The MUX32FU module is a 4-to-1 multiplexer that selects one of four 32-bit inputs based on a 2-bit selector. It is used as the multiplexer in the two forwarding paths that feed the ALU inputs. The selection logic is as follows:
 - 00: Selects the register value from the ID/EX pipeline, no hazard.
 - 01: Selects the write-back data from the MEM/WB stage to resolve a MEM hazard.
 - 10: Selects the ALU result from the EX/MEM stage to resolve an EX hazard.
 The selector bits are determined by the output of the Forwarding_Unit, which detects hazards and determines the appropriate forwarding source.
- **Sign_Extend.v**
The Sign_Extend module takes a 12-bit immediate as input and sign-extends it to 32 bits. It does so by copying the sign bit (the 11th bit) into the upper 20 bits. This allows the immediate to be correctly interpreted as a signed 32-bit value during ALU operations.

Difficulties encountered and solutions in this lab

I spent most of my time debugging a dumb mistake I made while typing. Since Forwarding Units require a larger multiplexer supporting four inputs and a two-bit selector, I naively copied the old version of the MUX32 module and added two more inputs, then forgot to adjust the size of the selector bits. My program didn't crash even though the data I input was two bits for the selector register. Since the program ran and even passed the first test case, I was so confused about what went wrong. Then I used GTKWave to figure out the exact part that went wrong. I discovered that the output of my x16 register was wrong at cycle 6, so I traced it back to where the data was created—all the way to the ALU module's inputs—and found that the output of the Forward A MUX was wrong! So I went back to the module, checked again, and finally discovered and fixed this stupid mistake that took me quite some time to debug. After that problem was solved, the system ran smoothly without any errors (at least based on the public test cases).

Development environment

Developed on a macOS system using Verilog. Code was written and edited using Visual Studio Code, then ran the simulation on Jupiter. Debugged by using the waveform visualization tool gtkwave.